

Technical Report: Disk Scheduling Algorithm Visualizer

Situation: During the development of the Disk Scheduling Visualizer, the initial implementation of the FCFS (First-Come-First-Served) and SSTF (Shortest Seek Time First) algorithms was straightforward, as they solely rely on calculating absolute distances between the current head position and the next request. However, when I began implementing the SCAN (Elevator) and C-SCAN (Circular SCAN) algorithms within the **algorithms.py** module, the simulation produced highly inaccurate total seek times. Furthermore, when the data was passed to the **visualizer.py** module, the resulting graphs were erratic and did not reflect realistic disk arm movement. The core issue was handling what happens when the disk head reaches the physical boundaries of the disk (Track 0 and Track 199).

Task: My objective was to accurately simulate the physical constraints of a mechanical disk arm. I needed to ensure that when the SCAN algorithm reached the maximum disk limit, it properly reversed direction, and when C-SCAN reached the limit, it properly jumped to zero. Most importantly, I had to ensure these boundary conditions were mathematically accounted for in the total seek time calculation, and visually represented as sharp turns or drops on the Matplotlib graphs.

Action: Instead of attempting to sort and calculate the path in a single, complex loop—which was causing the bugs—I refactored the code by breaking the request queue into two distinct arrays. I used Python list comprehensions to separate the incoming requests into a left array (tracks smaller than the initial head) and a right array (tracks larger than the initial head).

I then applied conditional sorting to these arrays: descending for the **left** array in SCAN, and ascending for the **left** array in C-SCAN. The critical fix was explicitly appending the physical disk boundaries (e.g., **size - 1** and **0**) directly into the path arrays before calculating the total distance. Finally, in **visualizer.py**, I utilized Matplotlib's **plt.step(where='post')** function instead of a standard line plot, which accurately renders the discrete jumps and sharp directional shifts at the disk boundaries, rather than drawing diagonal lines between distant tracks.

Result: This modular approach completely resolved the mathematical and visual bugs. The seek times generated by the program now perfectly match theoretical textbook examples for all four algorithms. The graphical output successfully and clearly visualizes the OS concepts: the 2x2 grid comparison clearly shows the head sweeping to the edge of the disk in SCAN, and dramatically dropping to zero in C-SCAN. This architectural fix not only made the project function correctly but also made the abstract concept of disk scheduling instantly understandable through the visual output.